

Find My Mines

An aim of this assignment is to demonstrate how to implement basic socket programming in client-server applications. The **Find My Mines** game must be implemented using socket programming as a two-player (or multi-player) online game. The game client must be deployed on at least two computers, allowing users to play the game between two or more connected computers as a regular online application.

Client-Server

A server must be implemented to wait for client connections. When a client is on the network, a client would connect to the server first. Then, the server will provide information about the other connected client. This allows each client to know which users are currently connected to the same network.

The setup has one computer to run both the server program and a game client, while another computer runs only the game client, which connects directly to the server.



Requirement

- The game client and server must communicate via **socket programming**, based on a **client-server** model.
- Assume each client game knows the server's address and port. Game clients **do not need to enter** the IP address or port number. The server's IP address and port must be set in the program's source code.
- The server program must display: (1) the number of concurrent clients currently connected (2) a list of those connected clients.
- Each player enters a nickname before the game starts. When the game starts, a welcome message with their nickname will appear, such as:

"Welcome, Alice."

- The server randomly places **11 bombs** on a **6×6 grid**.
- The player's name and score are displayed on the game client.

- The server randomly selects the first player for the first match.
- At the start, all slots are covered.
- Each player has **10 seconds per turn** to find bombs. If a bomb is found, the player **continues their turn** until time runs out; otherwise, the turn passes to another player.
- Selected slots are marked as “**bomb**” or “**empty**” slots.
- If the selected slot is **empty**, it displays the total number of bombs in its eight surrounding slots (including diagonals) and is then disabled.
- Players earn **1 point** for each **bomb** slot they have found.
- The match ends when all bombs have been found.
- When the match ends, the game clients display the player’s status as “**Win**” or “**Lost**” along with the current scores of both players and a **Rematch** button.
- If both players agree to a rematch, the winner of the previous match becomes the first player.
- The server has a reset button to reset the game and players’ scores.

Example User Interface



Note that you can implement in any programming language and make your own creative design.

Hint:

https://www.tutorialspoint.com/java/java_socket_programming.htm

<http://java.sun.com/developer/onlineTraining/Programming/BasicJava2/socket.html>

<https://nodejs.org/api/net.html>

<https://socket.io/docs/v4/>

<https://docs.python.org/3/library/socket.html>

<https://docs.aws.amazon.com/apigateway/index.html>

<https://www.vpn.net>

<https://aws.amazon.com/getting-started/aws-networking-essentials/>

Find My Mines Score Criteria (Full score = 25 points)

a. Demo preparation and creativity (5 points)

Creativity in game design and demonstration on the submission day will be considered.

b. Fundamental implementation (10 points)

A server program and a game client must be implemented using **socket programming**.

System Setup

- (1.0) All requirements have been **completed**.
- (0.5) One computer runs both the server program and game client.
- (0.5) One computer runs only the game client, which connects directly to the server.

Client

- (0.5) The client connects to the server without requiring users to manually enter the IP address or port number.
- (1.0) The client connects to the server first and receives information about other connected clients.
- (0.5) Each player enters a nickname and sees a welcome message.
- (0.5) The player's name and score are displayed on the game client.
- (0.5) A **10-second countdown** timer runs during each player's turn.
- (0.5) Each selected grid slot is **disabled**.
- (0.5) Selected slots are marked as "**bomb**" or "**empty**" slots.
- (0.5) The player's score is updated according to the requirements. When the match ends, the player's **status** and current scores are displayed.
- (0.5) Both players can rematch.

Server

- (1.0) The server program displays the number of clients currently connected (online) and a list of connected clients.
- (1.0) The server has a reset button to reset the current game and players' scores.
- (0.5) The server randomly places **11 bombs** on a **6×6 grid** and randomizes the first player.
- (0.5) The server assigns the winner from the last match to start first in the **rematch**.

c. Extra features (Maximum 10 points)

- AI feature: 2 points. **At least one AI feature is required.**
- Non-AI feature: 1 point

If a feature changes the existing game design, it may be implemented as a separate **mode**.

Extra features are not considered if the fundamental implementation is incomplete.

Feature name	AI Feature	Feature name	AI Feature
1.	✓	6.	
2.		7.	
3.		8.	
4.		9.	
5.		10.	